

Database Normalization Guide

Subject: Database Management Systems Topic: Normalization

Overview

Step-by-step normalization from 1NF to BCNF with practical examples

Why normalize?

- Normalization organizes relational data to reduce duplicate facts and prevent insert, update, and delete anomalies.
- A functional dependency $X \rightarrow Y$ means each X value determines exactly one Y value.
- Decompose tables carefully and preserve keys and dependencies where possible.

Normal forms

- 1NF requires atomic column values and rows identifiable by a key; do not store repeating groups in one field.
- 2NF requires 1NF and every non-key attribute to depend on the whole candidate key, not a proper subset.
- 3NF requires 2NF and removes transitive dependencies of non-key attributes on a key.
- BCNF requires every determinant in a nontrivial functional dependency to be a superkey.

Example

- If Enrollment(student_id, course_id, student_name, grade) uses a composite key and student_name depends only on student_id, it violates 2NF.
- Move student attributes to Student(student_id, student_name); keep course-specific grade in Enrollment.
- The decomposition avoids repeating a student's name for each course and keeps updates consistent.

Design trade-offs

- Use foreign keys and constraints to enforce relationships and valid references.
- Normalization is a correctness baseline; denormalization may be justified for measured read performance.
- Document the source of duplicated values and keep them synchronized if denormalization is used.